Binary Tree Diameter

AlgoExpert · Medium · Binary Trees

The problem

- Number of edges on the longest path between any two nodes.

Intuition

- The longest path through a node joins the heights of its two subtrees.

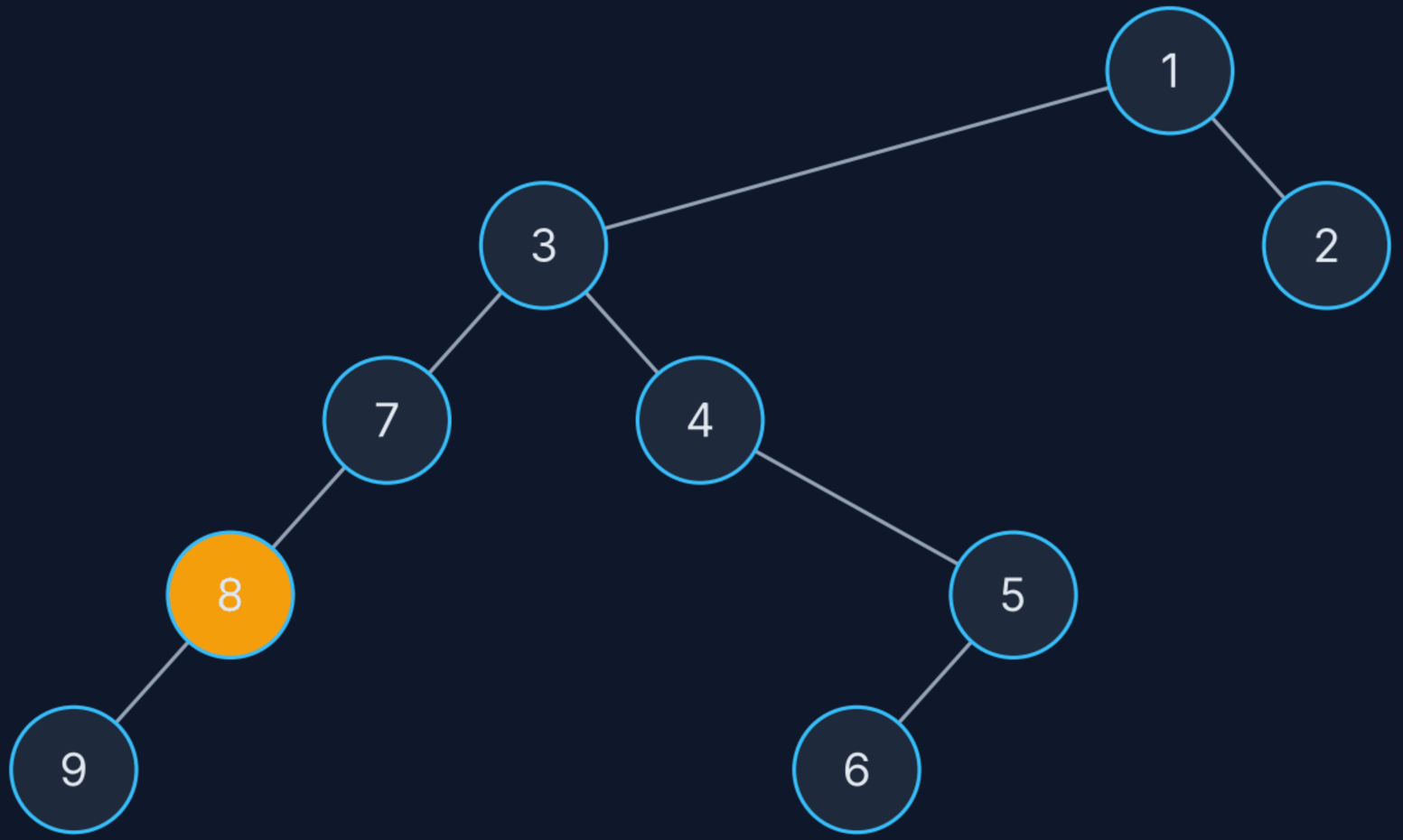
Brute force vs optimal

- Brute force: diameter through each node via separate height calls → $O(n^2)$
- Post-order computing height once per node → $O(n)$
- Diameter need not pass through the root

Approach

- 1. Post-order returns each subtree's height
- 2. At each node, left height plus right height is a candidate
- 3. Track the maximum

Worked example



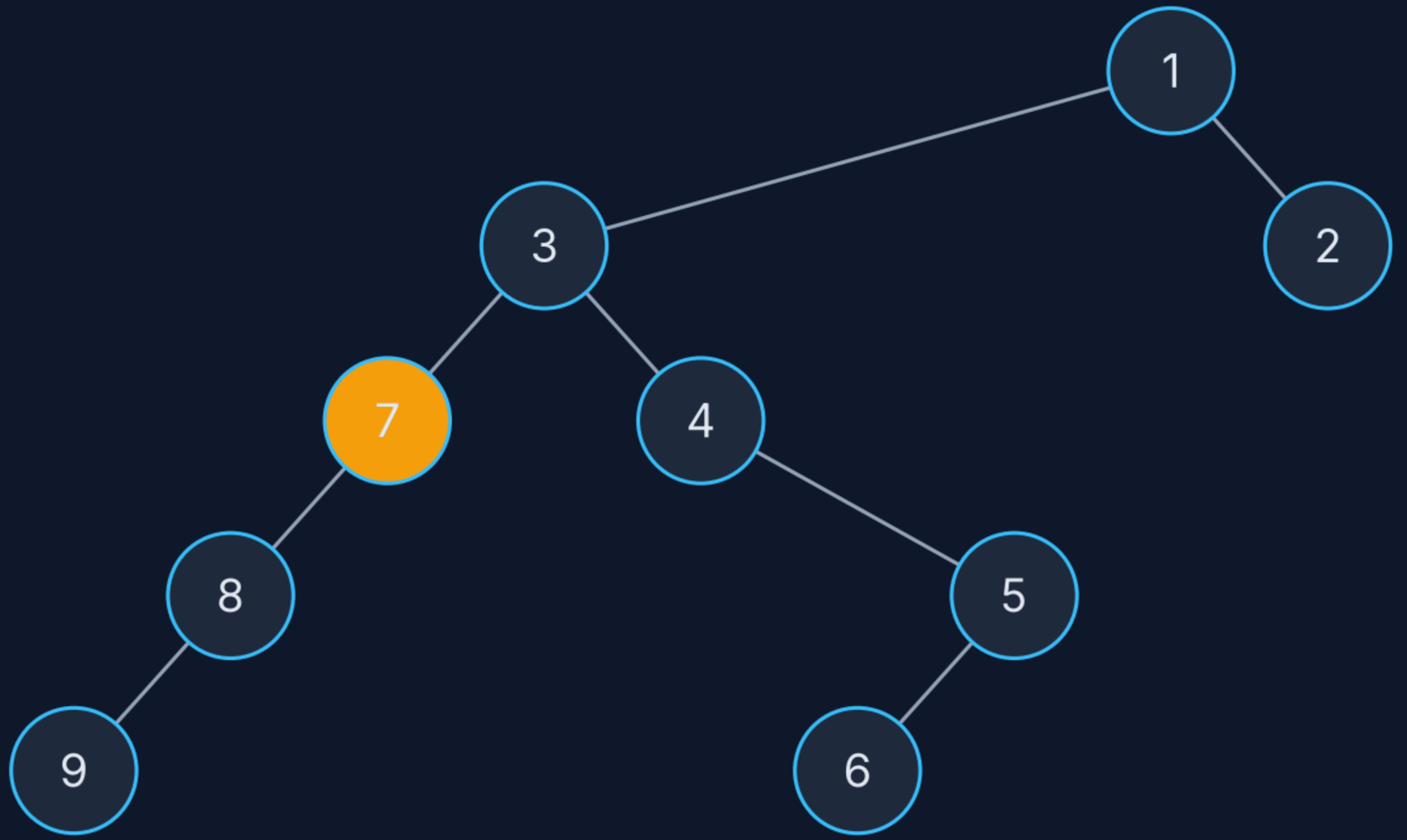

```
h(left) = 1

h(right) = 0

best = 1
```

At 8: left 1 + right 0 = 1; best 1

Worked example



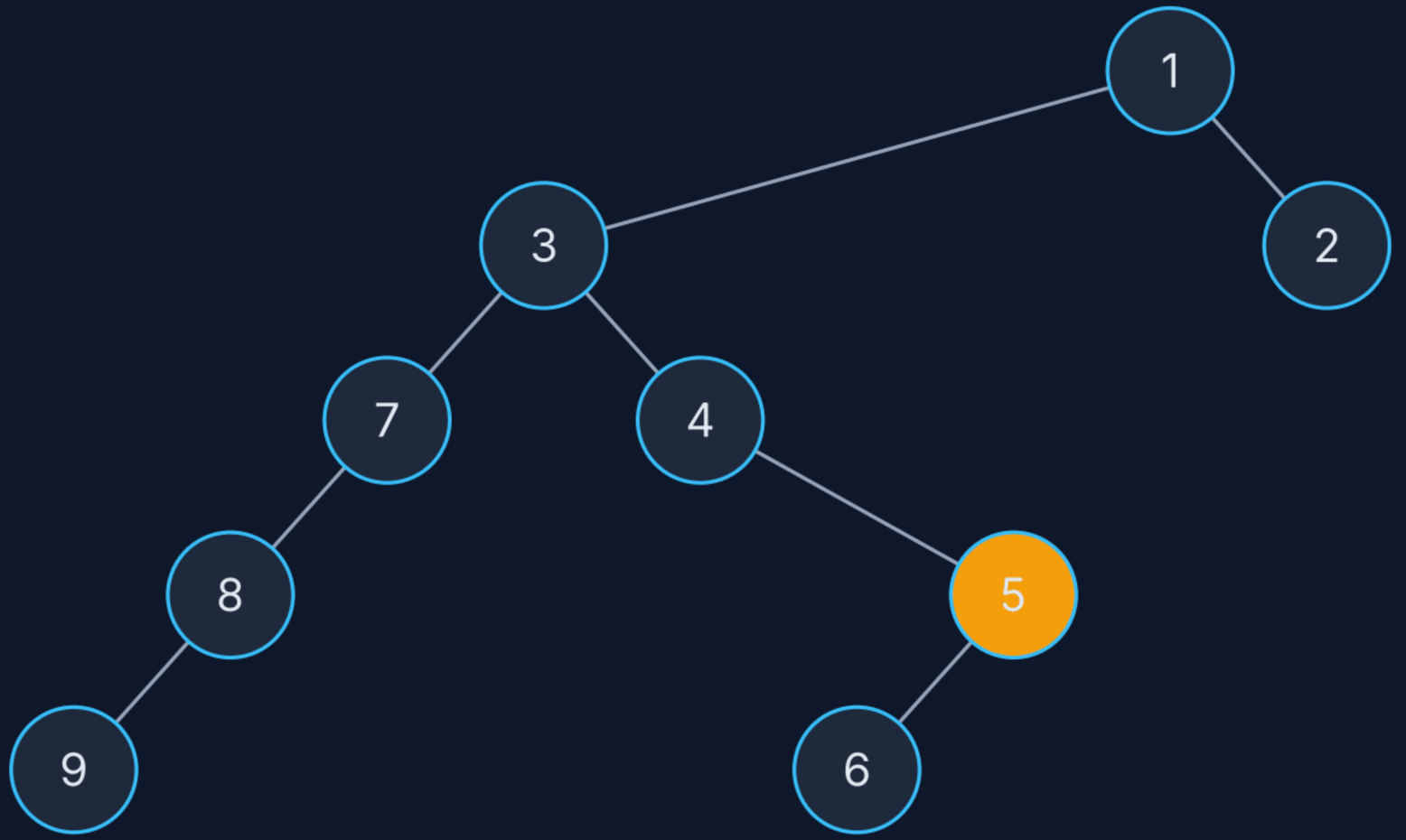

```
h(left) = 2

h(right) = 0

best = 2
```

At 7: left 2 + right 0 = 2; best 2

Worked example



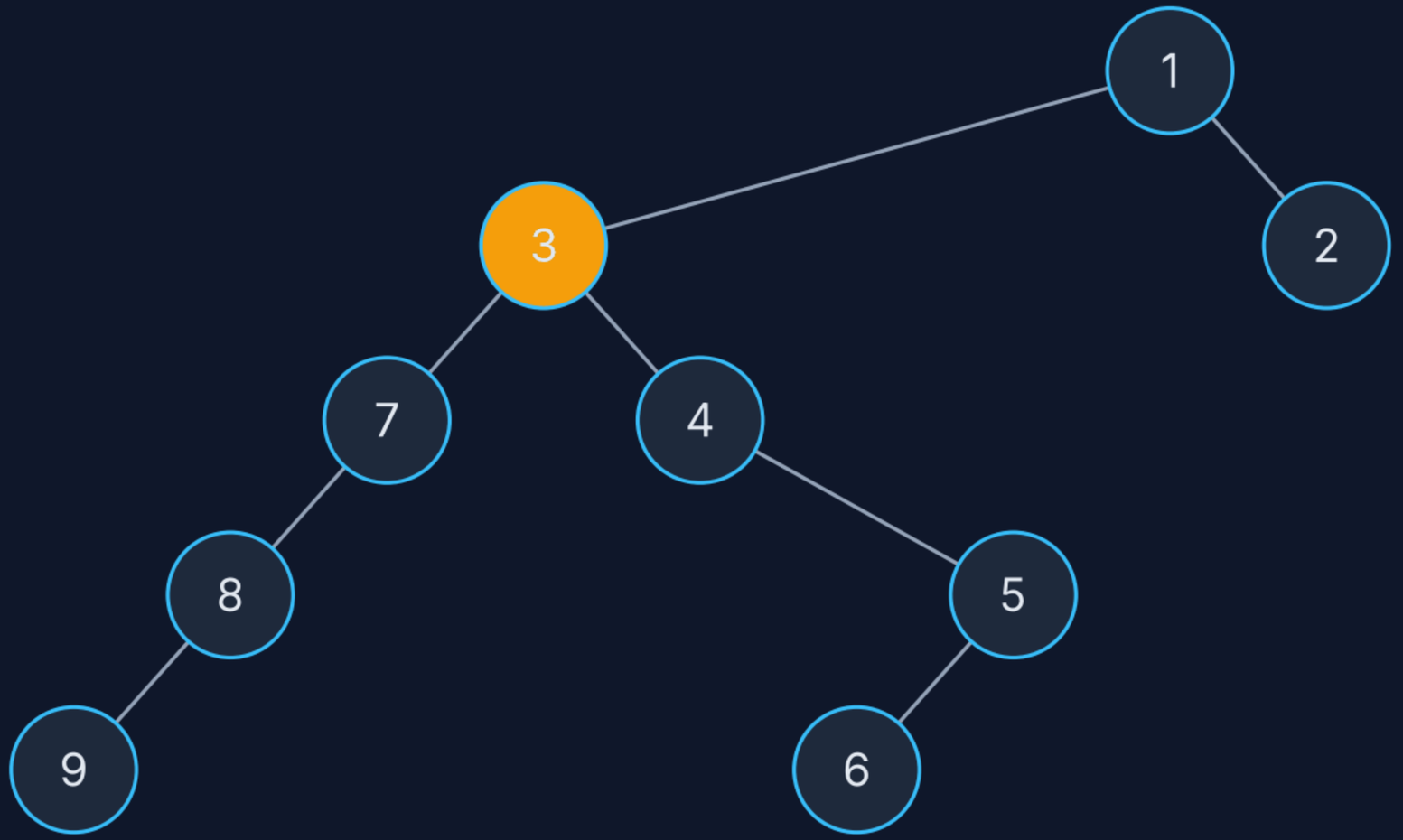

```
h(left) = 1

h(right) = 0

best = 2
```

At 5: left 1 + right 0 = 1; best 2

Worked example



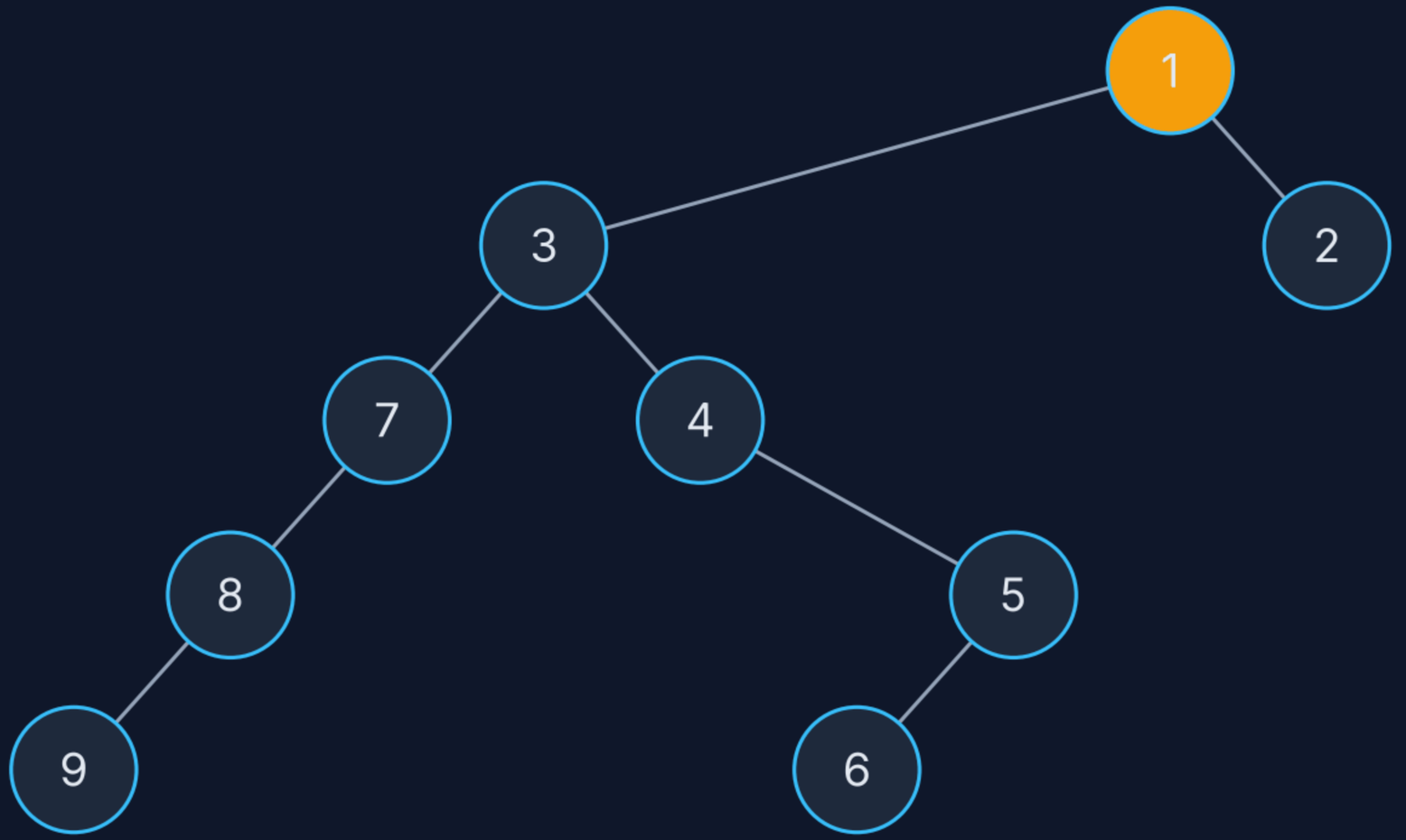

```
h(left) = 3

h(right) = 3

best = 6
```

At 3: left 3 + right 3 = 6; best 6

Worked example




```
h(left) = 4

h(right) = 1

best = 6
```

At 1: left 4 + right 1 = 5; best 6

Java solution

```java
public class BinaryTreeDiameter {
    public static int binaryTreeDiameter(BinaryTree t) {
        int[] best = {0};
        height(t, best);
        return best[0];
    }

    private static int height(BinaryTree t, int[] best) {
        if (t == null) return 0;
        int l = height(t.left, best), r = height(t.right, best);
        best[0] = Math.max(best[0], l + r);
        return 1 + Math.max(l, r);
    }
}
```

Complexity

- Time: O(n)
- Space: O(h)

Edge cases & pitfalls

- Diameter counts edges, not nodes
- Longest path may avoid the root
- Single node → 0

Interview tips & follow-ups

- Return height, track best in a field/array
- Follow-up: max path SUM (Hard)

Recap

- The longest path through a node joins the heights of its two subtrees.
- Complexity: O(n)